

Safe Direct Access

There are certain circumstances where it seems reasonable to access instance variables directly: when it is absolutely clear what the instance variable means, little or no validation of the data is needed, and there is no chance that the name will ever change. A good example of this is the `Rect` (rectangle) class in the `pygame` package. A rectangle in `pygame` is defined using four values—`x`, `y`, `width`, and `height`—like this:

```
oRectangle = pygame.Rect(10, 20, 300, 300)
```

After creating that rectangle object, using `oRectangle.x`, `oRectangle.y`, `oRectangle.width`, and `oRectangle.height` directly as variables seems acceptable.

Making Instance Variables More Private

In Python, all instance variables are public (that is, can be accessed by code external to the class). But what if you want to allow access to some of your class's instance variables, but not all of them? Some OOP languages allow you to explicitly mark certain instance variables as `public` or `private`, but Python doesn't have those keywords. However, there are two ways that programmers who develop classes in Python can indicate that their instance variables and methods are intended to be private.

Implicitly Private

To mark an instance variable as one that should never be accessed externally, by convention you start the name of your instance variable with one leading underscore:

```
self.__name  
self.__socialSecurityNumber  
self.__dontTouchThis
```

Instance variables with names like these are intended to represent private data, and client software should never attempt to access them directly. The code may still work if the instance variables are accessed, but it is not guaranteed.